

Lecture 3: Merge Sort & Quick Sort

*Instructor: Sourav Chakraborty, Sanjukta Roy**Scribe: Suman, Madhumita*

1 Divide and Conquer

Divide and Conquer is an algorithmic technique where a problem is:

- Divided into smaller subproblems
- Each subproblem is solved recursively
- The solutions are combined

1.1 Sorting

Input: Sorting input n numbers

Output: The numbers in sorted order

1.2 Examples of Divide and Conquer Sorting

- Quick Sort
- Merge Sort

2 Merge Sort

2.1 Problem Definition

Input: A set of n numbers

Output: The numbers in sorted order

2.2 Algorithm Idea

- Recursively divide the array into two halves
- Continue dividing until singleton elements are obtained
- Singleton elements are by default sorted
- Merge the sorted subarrays efficiently

2.3 Recurrence Relation (Worst Case)

$$T(n) = 2T\left(\frac{n}{2}\right) + cn$$

(each problem is divided into two subproblems of size $n/2$)

$$T(n) = cn \log n = O(n \log n)$$

2.4 Complexity Analysis

- Worst case time complexity: $O(n \log n)$
- Best case time complexity: $O(n \log n)$
- Space complexity: $O(n)$ (extra space)

2.5 Properties

- Not an in-place sorting algorithm
- Stable i.e. equal elements keep their relative order intact
- Even if the array is already sorted, Merge Sort takes $O(n \log n)$ time.

2.6 Merging Process

Let two sorted subarrays $a_1[i]$ and $a_2[j]$ be merged into array $a[k]$.

$$a[k] = \min(a_1[i], a_2[j])$$

If $a_1[i] < a_2[j]$, increment i , else increment j .

Increment k in both cases.

3 Quick Sort

3.1 Algorithm Idea

- Pick a pivot element
- Break the array into two subarrays such that:
 - Left subarray contains elements $<$ pivot
 - Right subarray contains elements $>$ pivot
- Recursively apply Quick Sort to each subarray

The size of the subarrays may or may not be equal (compare this step with Merge Sort!).

3.2 Running Time Analysis

$$T(n) = T(a) + T(n - a - 1) + cn$$

(where the position of the pivot is unknown)

3.3 Worst Case

Occurs when the pivot is the smallest or the largest element:

$$T(n) = T(n - 1) + cn = O(n^2)$$

3.4 Best Case

Occurs when pivot is the median:

$$T(n) = 2T\left(\frac{n}{2}\right) + cn = O(n \log n)$$

In this case, the pivot divides the array into equal length subarrays.

3.5 Average Case

- Expected time complexity is $O(n \log n)$
- In-place sorting (preferred in practice)
- Stability depends on implementation

To get best case behaviour each time, we can spend at most $O(n)$ time finding the median. Can we do better than $O(n \log n)$ sorting?

4 Lower Bound for Sorting (Decision Tree Model)

4.1 Decision Tree

A decision tree is a tree structure where:

- Each internal node represents a decision/comparison
- Each leaf represents a sorted output

Decisions in sorting algorithms:

$$a > b \mid a < b \mid a = b \mid a \leq b \mid a \geq b$$

Assuming all elements are distinct:

$$a > b \mid a < b$$

4.2 Leaves of Decision Tree

In the leaves, we obtain sorted arrays.

$$\text{Number of leaves} = n!$$

An execution of a sorting algorithm follows a path from the root to a leaf.

If the algorithm is correct for all possible inputs, there must be $n!$ leaves.

4.3 Height of Decision Tree

If the height of the decision tree is h and as the number of nodes doubles at each level:

$$\text{Then number of leaves} = 2^h$$

Now,

$$2^h \geq n! \Rightarrow h \geq \log_2(n!) = \sum_{i=1}^n \log i$$

Using Stirling's approximation:

$$h = \Omega(n \log n)$$

$$n! = n(n-1)(n-2) \dots (n/2)(n/2-1) \dots 2.1$$

$$\geq (n/2)^{n/2} (n/2-1) \dots 2.1$$

$$\geq (n/2)^{n/2}$$

$$\Rightarrow \log(n!) \geq \log(n/2)^{n/2} = \Omega(n \log n)$$

4.4 Theorem

Any comparison-based sorting algorithm takes at least $\Omega(n \log n)$ time in the worst case.

5 Count-Based Sorting

Count-based sorting uses extra space and counts the frequency of each element.

5.1 Example

Input: $A[] = \{5, 3, 7, 2, 5, 3\}$

Frequency array:

Index	1	2	3	4	5	6	7
Frequency	0	1	2	0	2	0	1

Using array for sake of understanding. Array may not be the best Data Structure for this
(In case of range of the elements $\gg n$)

5.2 Algorithm

Algorithm 1 Count Based Sorting

```
1: procedure COUNTSORT(Array  $A$ ,  $n$ )
2:   Initialize frequency array  $freq$ 
3:   for  $i = 0$  to  $n - 1$  do
4:      $freq[A[i]] \leftarrow freq[A[i]] + 1$ 
5:   end for
6:   Recreate Sorted Array from  $freq$ 
7: end procedure
```

5.3 Complexity

- Time complexity: $O(n + m)$

- Space complexity: $O(m)$, where m is the largest element

If freq is implemented using an array, required space is:

$$\max(A[i]) - \min(A[j]), \quad i, j \in \{1, \dots, n\}$$

5.4 Exercise

If freq is implemented using a hash table:

- What is the size of the hash table?
- How many times does step 4 of the algorithm execute?

6 Remarks

- In-place sorting is preferred for large data (e.g. databases)
- Quick Sort is widely used due to cache efficiency